PRACTICAL: 02

Implement breadth-first search, depth-first search and uniform-cost search on graph or maze problems; compare search cost and path quality.

Aim: To implement Breadth-First Search (BFS), Depth-First Search (DFS), and UniformCost Search (UCS) on a graph or maze problem and compare their search cost, path quality, and efficiency.

Theory: An Artificial Intelligence (AI) agent is a system that perceives its environment and takes actions to achieve a specific objective. An agent receives information about the environment through sensors and performs actions using actuators.

Program / Code:

1. Breadth-First Search:

```
print("Hello! I am your Treasure Robot")
print("Let's find the treasure together!")
map_rooms = {
   "A": ["B", "C"],
   "B": ["D"],
   "C": ["D"],
   "D": ["G"],
   "G":      []      }
print("Our  map:")
print("   A (Start)")
print("        /    \\")
print("   B         C")
print("        \\    /")
print("   D")
```

```
print("    |") print("    G (Treasure)") print()
print("Doors from each room:", map_rooms)
def bfs(start,
goal):    to_do =
[start]    visited =
[]    order = []
while to_do:
      room = to_do.pop(0)
if room in visited:
continue
visited.append(room)
order.append(room)
if room == goal:
return order
      for next_room in map_rooms[room]:
         to_do.append(next_room)    return order
bfs_order = bfs("A", "G") print("BFS checked the
rooms in this order:", bfs_order) print("Number of
rooms BFS checked:", len(bfs_order))
```

Output:

```
Hello! I am your Treasure Robot
Let's find the treasure together!
Our map:
   A (Start)
  / \
 B   C
  \ /
   D
   |
   G (Treasure)

Doors from each room: {'A': ['B', 'C'], 'B': ['D'], 'C': ['D'], 'D': ['G'], 'G': []}
BFS checked the rooms in this order: ['A', 'B', 'C', 'D', 'G']
Number of rooms BFS checked: 5

=== Code Execution Successful ===
```

2. Depth-first Search:

```
print("Hello! I am your Treasure Robot") print("Let's
find the treasure together!")

map_rooms = {
   "A": ["B", "C"],
   "B": ["D"],
   "C": ["D"],
   "D": ["G"],
   "G": []
}

print("Our        map:")
print("       A  (Start)")
print("   / \\") print(" B
C")   print("      \\   /")
print("   D") print("   |")
print("   G (Treasure)")
print()

print("Doors from each room:", map_rooms)


def dfs(start, goal):
to_do = [start]     visited
= []
```

```
    order = []
while to_do:
        room = to_do.pop()
if room in visited:
            continue
visited.append(room)
order.append(room)          if
room == goal:
return order

        for next_room in map_rooms[room]:
            to_do.append(next_room)

    return order

dfs_order = dfs("A", "G")

print("DFS checked the rooms in this order:", dfs_order) print("Number
of rooms DFS checked:", len(dfs_order))
```

Output:

```
Hello! I am your Treasure Robot
Let's find the treasure together!
Our map:
   A (Start)
  / \
 B   C
  \ /
   D
   |
   G (Treasure)

Doors from each room: {'A': ['B', 'C'], 'B': ['D'], 'C': ['D'], 'D': ['G'], 'G': []}
DFS checked the rooms in this order: ['A', 'C', 'D', 'G']
Number of rooms DFS checked: 4

=== Code Execution Successful ===
```

3. compare search cost and path quality.

```
door_cost = {
   ("A", "B"): 1,
   ("B", "D"): 1,
   ("A", "C"): 5,
   ("C", "D"): 1,
   ("D", "G"): 1 }
def
path_cost(path):
   total = 0     for i in
range(len(path) - 1):
       total += door_cost[(path[i], path[i + 1])]
return total path1 = ["A", "B", "D", "G"]
path2 = ["A", "C", "D", "G"] cost1 =
path_cost(path1) cost2 = path_cost(path2)
print("Path 1 (A -> B -> D -> G) cost:",
cost1) print("Path 2 (A -> C -> D -> G)
cost:", cost2) if cost1 < cost2:     best_path =
path1     best_cost = cost1 else:
   best_path = path2     best_cost
= cost2 print("Cheapest path:",
best_path) print("Total cost:",
best_cost)
```

Output:

```
Path 1 (A -> B -> D -> G) cost: 3
Path 2 (A -> C -> D -> G) cost: 7
Cheapest path: ['A', 'B', 'D', 'G']
Total cost: 3

=== Code Execution Successful ===
```

Conclusion: BFS, DFS, and UCS provide different approaches to solving graph and maze problems. BFS is suitable when all actions have equal cost and the shortest path is required. DFS can reach a solution quickly and uses less memory but does not guarantee the best path. UCS is the best choice when paths have different costs because it guarantees the least-cost solution under appropriate conditions. Therefore, the choice of search algorithm depends on the graph structure, path costs, memory limitations, and required path quality.